



Basic Input/Output

- [https://legacy.cplusplus.com/doc/tutorial/basic_io/]

The example programs of the previous sections provide little interaction with the user, if any at all. They simply printed simple values on the screen, but the standard library provides many additional ways to interact with the user's via its input/output features. This section will present a short introduction to some of the most useful.

C++ uses a convenient abstraction called *streams* to perform input and output operations in sequential media such as the screen, the keyboard or a file. A *stream* is an entity where a program can either insert or extract characters to/from. There is no need to know details about the media associated to the stream or any of its internal specifications. All we need to know is that streams are a source/destination of characters, and that these characters are provided/accepted sequentially (i.e. one after another).

The standard library defines a handful of stream objects that can be used to access what are considered the standard source and destinations of characters by the environment where the program runs:

stream	description
<code>cin</code>	standard input stream
<code>cout</code>	standard output stream
<code>cerr</code>	standard error (output) stream
<code>clog</code>	standard logging (output) stream

We are going to see in more detail only `cout` and `cin` (the standard output and input stream); `cerr` and `clog` are also output streams, so they essentially work like `cout`, with the only difference being that they identify streams for specific purpose: error messages and logging; which, in many cases, in most environment setups, they actually do the exact same thing: they print on screen, although they can also be individually redirected.

在前面几节给出的示例代码里很少出现和用户的交互，if any at all(如果有的话)。前面几节的代码仅仅能在屏幕上打印一些简单的值，不过C++的标准库提供了更多能够用于用户交互的IO特

性。这一节我们讲简要的介绍它们中最为实用的一些。

C++使用一个叫做“流”的抽象来很方便的表现线型媒介上的输入输出操作(如屏幕、键盘或文件)。一个流是一个实体，对于这个实体一个程序可以在其中插入或者提取字符。(有了这个抽象，)你完全没有必要去了解“流”底下具体的媒介或是“流”本身的一些内在特性。全部要你了解的就只是流是一个提供串行字符的源点，或是接收串行字符的终点。（串行的意思是它们像排队一样，一个接一个的往前走。）

标准库定义了一下几种流来作为程序运行时操作字符的抽象(发送起点/接收终点):

流	描述
<code>cin</code>	标准输入流
<code>cout</code>	标准输出流
<code>cerr</code>	标准报错(输出)流
<code>clog</code>	标准日志(输出)流

我们将更多的关注 `cout` 和 `cin`，`cerr` 和 `clog` 可以看作具有特定用途的 `cout`，在实际使用中作为输出流和 `cout` 发挥着同样重要的作用。

○Standard output (cout)

On most program environments, the standard output by default is the screen, and the C++ stream object defined to access it is `cout`.

For formatted output operations, `cout` is used together with the *insertion* operator, which is written as `<<` (i.e., two "less than" signs).

```
cout << "Output sentence";    //prints Output sentence on screen
cout << 120;                  //prints number 120 on screen
cout << x;                    //prints the value of x on screen
```

The `<<` operator inserts the data that follows it into the stream that precedes it. In the example above, it inserted the literal string `Output sentence`, the number `120`, and the value of variable

`x` into the standard output stream `cout`. Notice that the sentence in the first statement is enclosed in double quotes(`"`) because it is a string literal, while in the last one, `x` is not. The double quoting is what makes the difference; when the text is enclosed between them, the text is printed literally; when they are not, the text is interpreted as the identifier or a variable, and its value is printed instead. For example, there two sentences have very different results:

```
cout << "Hello"; //prints Hello
cout << Hello;   //prints the content of variable Hello
```

Multiple insertion operations (`<<`) may be chained in a single statement:

```
cout << "This " << "is a " << "single C++ statement";
```

This last statement would print the text `This is a single C++ statement`. Chaining insertion is especially useful to mix literals and variables in a single statement:

```
cout << "I am " << age << " years old and my zipcode is " << zipcode;
```

Assuming the `age` variable contains the value 24 and the `zipcode` variable contains `90064`, the output of the previous statement would be:

```
I am 24 years old and my zipcode is 90064
```

What `cout` does not do automatically is add line breaks at the end, unless instructed to do so.

...

这节讲 `cout` 。

```
cout << "Output sentence"; //将 Output sentence 打印到屏幕上
cout << 120;               //在屏幕上打印数字120
cout << x;                 //在屏幕上打印变量x的值
```

`cout` 和 `<<` (插入运算符) 一起工作。 `<<` (插入运算符) 将跟在它后面的东西插入到输出流。如上面的例子。

接下来是讲对于文本串使用双引号引用。也就是说，被双引号包围的文本作为文本原样输出，否则的话这些文本会被作为标识符(如变量名)识别。

```
cout << "Output sentence";    //将 Output sentence 打印到屏幕上
cout << 120;                  //在屏幕上打印数字120
cout << x;                    //在屏幕上打印变量x的值
```

你可以使用多个 `<<` 来进行链式的连接，这些被连接的对象可以是字符串和标识符混合的。

```
cout << "这是" << "一个" << "简单的C++语句";
cout << "我" << age << "岁了， 我的多多号是" << ddcode;
```

你还需要用到 `\n` 和 `endl` 来换行。

Standard input(cin)

In most program environment, the standard input by default is the keyboard, and the C++ stream object defined to access is `cin`.

For formatted input operations, `cin` is used together with the extraction operator, which is written as `>>` (i.e., two "greater than" signs). This operator is then followed by the variable where the extracted data is stored. For example:

```
int age;
cin >> age;
```

The first statement declares a variable of type `int` called `age`, and the second extracts from `cin` a value to be stored in it. This operation makes the program wait for input from `cin`; generally, this means that the program will wait for the user to enter some sequence with the keyboard. In this case, note that the characters introduced using the keyboard are only transmitted to the program where the `ENTER` (or `RETURN`) key is pressed. Once the statement with the extraction operation on `cin` is reached, the program will wait for as long as needed until some input is introduced.

...

在大多数程序所在的环境，标准输入是指键盘。`cin` 是C++定义来针对键盘的输入流。

`cin` 和 提取操作符 `>>` 一起工作。`>>` 后跟着一个变量，`>>` 从 `cin` 中取出一个数据然后存储到这个变量中。

需要注意的是，在键盘输入的字符会等待有回车或是RETURN键输入的时候才一起被传输到程序。而程序在每次运行到一个 `cin` 的时候会一直等待从键盘到来的字符。

...